

Books–Authors Management System using Spring Boot

NAME: REDDYSHETTY RISHWI

BITS ID: 2024eb01925

INTRODUCTION

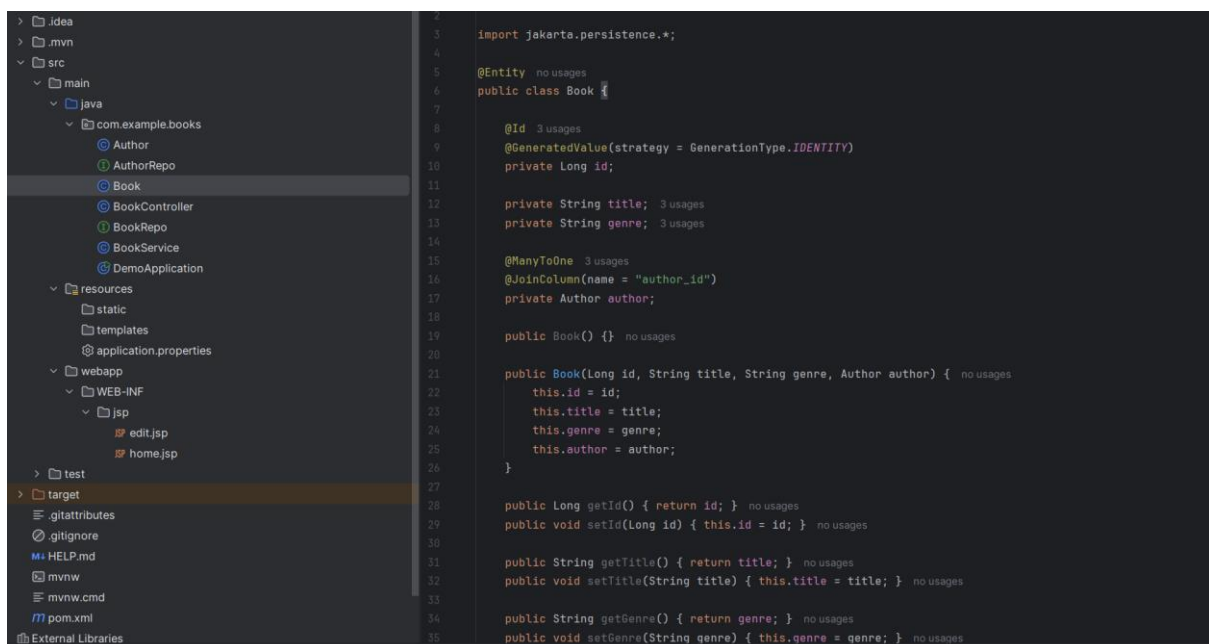
This project is a Books–Authors Management System developed using Spring Boot, JSP, and JPA. The application allows users to add, view, and update book records along with their associated authors. It demonstrates MVC architecture, database integration, and entity relationships.

ENTITY DESIGN

The system has two entities:

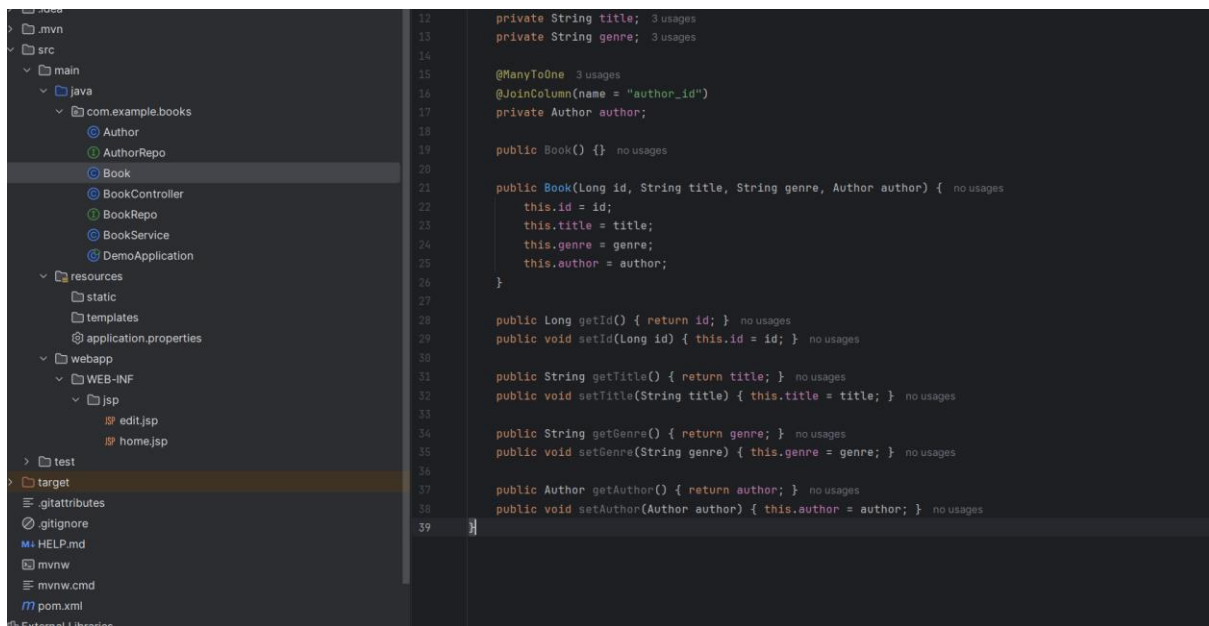
- **Book:** id, title, genre, author
- **Author:** id, name

A **Many-to-One relationship** is implemented where multiple books can belong to one author.



The screenshot displays an IDE with a project structure on the left and the source code of the `Book` entity on the right. The project structure includes a `main` package with `java` and `resources` sub-packages. The `java` package contains `com.example.books`, which includes `Author`, `AuthorRepo`, `Book`, `BookController`, `BookRepo`, `BookService`, and `DemoApplication`. The `resources` package contains `static`, `templates`, and `application.properties`. The `webapp` package contains `WEB-INF`, which includes `jsp` sub-package with `edit.jsp` and `home.jsp`. The `test` package is also visible. The `Book` entity code on the right is as follows:

```
1  import jakarta.persistence.*;
2
3  @Entity
4  public class Book {
5
6      @Id
7      @GeneratedValue(strategy = GenerationType.IDENTITY)
8      private Long id;
9
10     private String title;
11     private String genre;
12
13     @ManyToOne
14     @JoinColumn(name = "author_id")
15     private Author author;
16
17     public Book() {}
18
19     public Book(Long id, String title, String genre, Author author) {
20         this.id = id;
21         this.title = title;
22         this.genre = genre;
23         this.author = author;
24     }
25
26     public Long getId() { return id; }
27     public void setId(Long id) { this.id = id; }
28
29     public String getTitle() { return title; }
30     public void setTitle(String title) { this.title = title; }
31
32     public String getGenre() { return genre; }
33     public void setGenre(String genre) { this.genre = genre; }
34 }
```

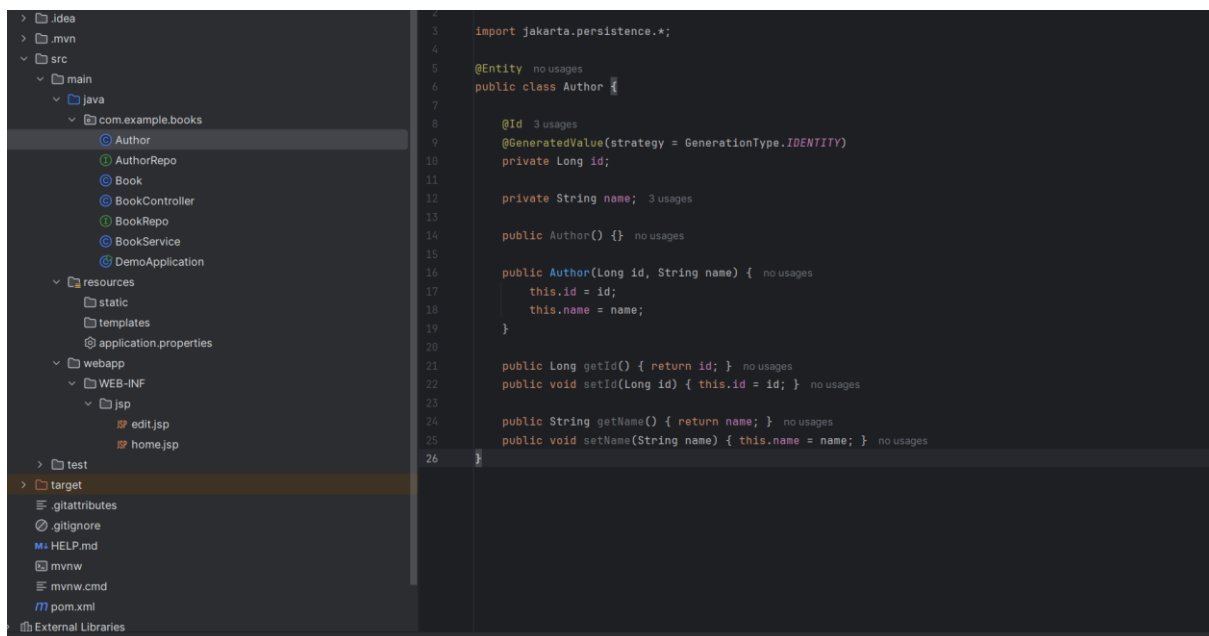


The screenshot shows an IDE with a project structure on the left and the `Book.java` file open in the editor. The project structure includes a `src/main/java/com.example.books` package with classes `Author`, `AuthorRepo`, `Book`, `BookController`, `BookRepo`, `BookService`, and `DemoApplication`. The `Book.java` file contains the following code:

```

12 private String title; 3 usages
13 private String genre; 3 usages
14
15 @ManyToOne 3 usages
16 @JoinColumn(name = "author_id")
17 private Author author;
18
19 public Book() {} no usages
20
21 public Book(Long id, String title, String genre, Author author) { no usages
22     this.id = id;
23     this.title = title;
24     this.genre = genre;
25     this.author = author;
26 }
27
28 public Long getId() { return id; } no usages
29 public void setId(Long id) { this.id = id; } no usages
30
31 public String getTitle() { return title; } no usages
32 public void setTitle(String title) { this.title = title; } no usages
33
34 public String getGenre() { return genre; } no usages
35 public void setGenre(String genre) { this.genre = genre; } no usages
36
37 public Author getAuthor() { return author; } no usages
38 public void setAuthor(Author author) { this.author = author; } no usages
39 }

```



The screenshot shows an IDE with a project structure on the left and the `Author.java` file open in the editor. The project structure is the same as the previous screenshot. The `Author.java` file contains the following code:

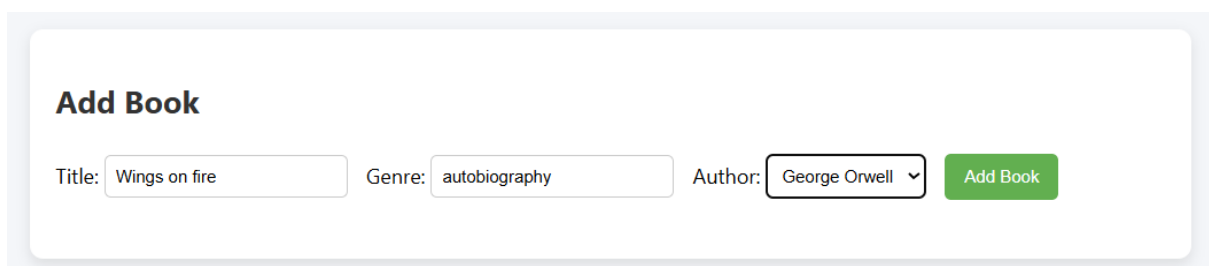
```

3 import jakarta.persistence.*;
4
5 @Entity no usages
6 public class Author {
7
8     @Id 3 usages
9     @GeneratedValue(strategy = GenerationType.IDENTITY)
10    private Long id;
11
12    private String name; 3 usages
13
14    public Author() {} no usages
15
16    public Author(Long id, String name) { no usages
17        this.id = id;
18        this.name = name;
19    }
20
21    public Long getId() { return id; } no usages
22    public void setId(Long id) { this.id = id; } no usages
23
24    public String getName() { return name; } no usages
25    public void setName(String name) { this.name = name; } no usages
26 }

```

CREATE OPERATION

Users can add a new book using a form by entering title, genre, and selecting an author. The data is sent to the controller and saved using the service layer.



The screenshot shows a web form titled "Add Book". It has three input fields: "Title" with the value "Wings on fire", "Genre" with the value "autobiography", and "Author" with a dropdown menu showing "George Orwell". There is a green "Add Book" button to the right of the "Author" field.

READ OPERATION

All books are displayed on the homepage in a table format with title, genre, and author name.

All Books

Title	Genre	Author	Action
Harry Potter	Fantasy	J.K. Rowling	Edit
1984	Dystopian	George Orwell	Edit
Animal Farm	Political	George Orwell	Edit
x1	autobiography	George Orwell	Edit
x2	political	J.K. Rowling	Edit
random 3	dystopian	George Orwell	Edit
testing 1	autobiography	J.K. Rowling	Edit
testing 3	political	J.K. Rowling	Edit
testing 5	autobiography	George Orwell	Edit
Wings on fire	autobiography	George Orwell	Edit

UPDATE OPERATION

Each book has an Edit option. Clicking it opens an edit page where details can be modified and saved.

localhost:8080/edit/10

Edit Book

Title:

Wings on fire

Genre:

political

Author:

J.K. Rowling

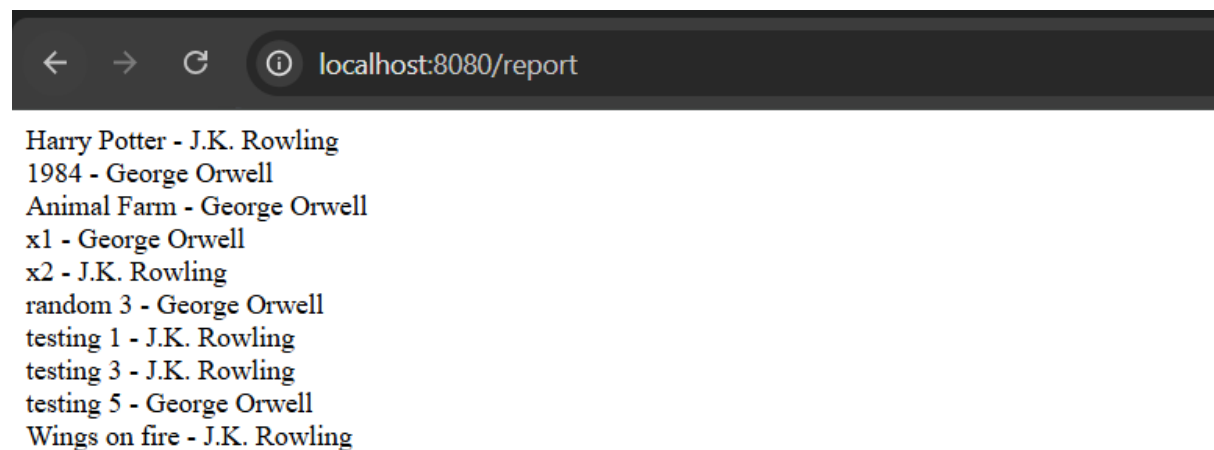
Update Book

All Books

Title	Genre	Author	Action
Harry Potter	Fantasy	J.K. Rowling	Edit
1984	Dystopian	George Orwell	Edit
Animal Farm	Political	George Orwell	Edit
x1	autobiography	George Orwell	Edit
x2	political	J.K. Rowling	Edit
random 3	dystopian	George Orwell	Edit
testing 1	autobiography	J.K. Rowling	Edit
testing 3	political	J.K. Rowling	Edit
testing 5	autobiography	George Orwell	Edit
Wings on fire	political	J.K. Rowling	Edit

CUSTOM JOIN QUERY

A custom query is used to fetch book titles and corresponding author names using an inner join.



ARCHITECTURE

The application follows MVC architecture:

- **Controller** → handles requests
- **Service** → business logic
- **Repository** → database interaction
- **Entity** → database tables

- > .idea
- > .mvn
- ▼ src
 - ▼ main
 - ▼ java
 - ▼ com.example.books
 - © Author
 - ⓘ AuthorRepo
 - © Book
 - © BookController
 - ⓘ BookRepo
 - © BookService
 - 🔄 DemoApplication
 - ▼ resources
 - static
 - templates
 - ⚙ application.properties
 - ▼ webapp
 - ▼ WEB-INF
 - ▼ jsp
 - JSP edit.jsp
 - JSP home.jsp
- > test

- > target
 - ≡ .gitattributes
 - 🚫 .gitignore
 - 📄 HELP.md
 - 📄 mvnw
 - ≡ mvnw.cmd
 - 📄 pom.xml

- > 📦 External Libraries

DATABASE

- H2 in-memory database is used
- Tables are auto-created using JPA
- Sample data inserted using CommandLineRunner

All Books

Title	Genre	Author	Action
Harry Potter	Fantasy	J.K. Rowling	Edit
1984	Dystopian	George Orwell	Edit
Animal Farm	Political	George Orwell	Edit
x1	autobiography	George Orwell	Edit
x2	political	J.K. Rowling	Edit
random 3	dystopian	George Orwell	Edit
testing 1	autobiography	J.K. Rowling	Edit
testing 3	political	J.K. Rowling	Edit
testing 5	autobiography	George Orwell	Edit
Wings on fire	political	J.K. Rowling	Edit

CHALLENGES

- Configuring JSP in Spring Boot
- Setting up database connection

GITHUB URL : <https://github.com/reddyshettyrishwi/sga-building-app>